Lab 3 – Containers & Docker

From Theory to Practice

Arquitectura de Xarxes

Universitat Pompeu Fabra

upf.

Outline

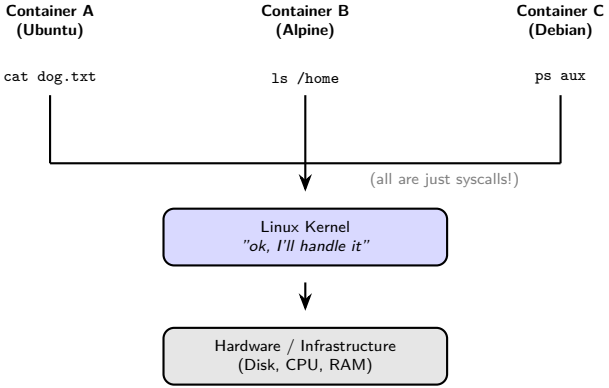
upf.

What Is a Container?

- A **container** is a lightweight, isolated environment that runs an application
- Unlike a Virtual Machine (VM), it does **not** include a full operating system
  - It shares the **host's kernel** and only packages the app and its dependencies

- Key characteristics:
  - **Fast**: starts in seconds (no OS to boot)
  - **Small**: measured in MBs, not GBs
  - **Portable**: same container runs identically on any machine with a container runtime
  - **Ephemeral**: designed to be created, destroyed, and replaced quickly

How Containers Share the Kernel



Every command inside any container is a syscall to the one shared kernel. The kernel uses **Linux namespaces** to ensure each container only sees its own files, processes, and network.

VMs vs Containers: Side by Side

Virtual Machines

- Include a **full guest OS** per instance
- Size: **GBs** (OS + app + dependencies)
- Boot time: usually **minutes**
- Strong isolation: each VM has its own kernel
- Can run **different operating systems**

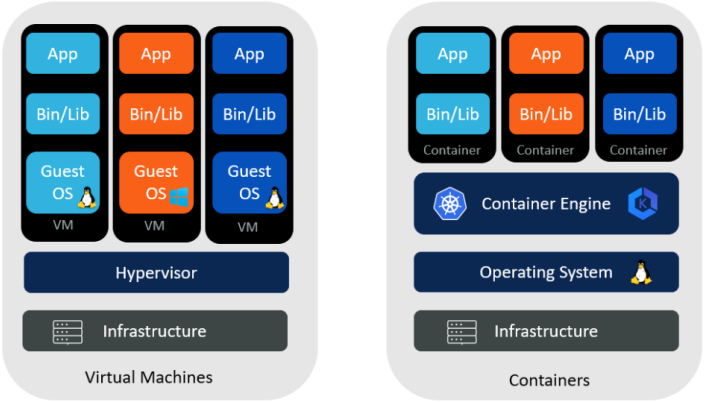
Containers

- **No guest OS**: share the host kernel
- Size: **MBs** (app + dependencies only)
- Boot time: usually **seconds**
- Shared kernel: a kernel vulnerability affects all
- Need a **container runtime** (e.g., Docker), not a hypervisor

Reminder: OS = Kernel + userspace (libraries, tools, shell, etc.).

upf.

VMs vs Containers: Visual Comparison



- **Bins/Libs** = binaries and libraries: the dependencies each app needs to run (e.g., Python, OpenSSL, libc)
- Containers share the same kernel; VMs each carry their own

Source: Aqueduct Technologies, "Containers and Virtual Machines," aqueducttech.com.

When to Use VMs vs Containers

- **Use VMs when:**
  - The app needs a **different OS** than the host (e.g., Windows app on Linux)
  - **Strong tenant isolation** is required (e.g., multi-tenant cloud)
  - Running **legacy software** tied to a specific OS version
- **Use containers when:**
  - Building cloud-native or **microservices** applications
  - Need for **fast scaling** (seconds, not minutes)
  - Working in Continuous Integration / Continuous Deployment (CI/CD) pipelines: build, test, and deploy in identical environments
- **In practice**: most modern platforms combine both
  - VMs provide the isolation layer; containers run inside them

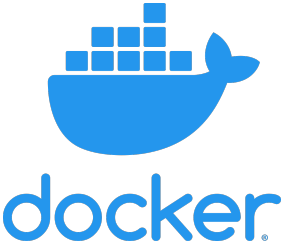
upf.

Outline

upf.

What Is Docker?

- **Docker** is the most widely used container platform (released 2013)
- It provides a complete toolchain to **build, ship, and run** containers
- Built on top of Linux kernel features: **namespaces** and **cgroups**
  - Namespaces: isolate what a container can *see* (filesystem, network, processes)
  - cgroups (control groups): limit what a container can *use* (CPU, RAM)
- Docker made containers **accessible**: simple CLI, image registry, standard format
- Before Docker, using namespaces and cgroups required manual low-level configuration



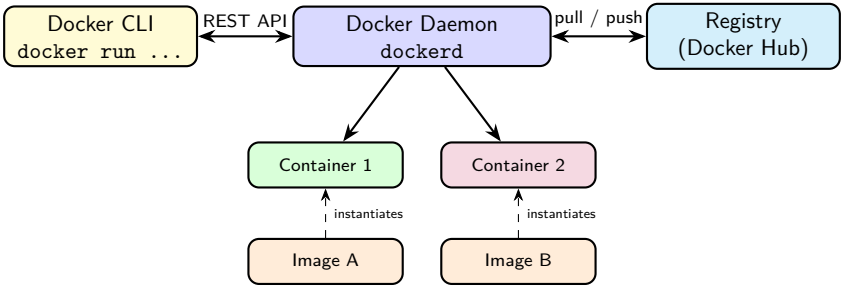
Source: Docker, Inc. docker.com

Key Docker Concepts

Concept	What it is
Image	Read-only template that defines the container (app + dependencies + config)
Container	A running instance of an image
Dockerfile	Text file with instructions to build an image
Registry	Storage for images (Docker Hub is the default public registry)
Volume	Persistent storage that survives container restarts
Network	Virtual network that connects containers to each other and to the outside

Analogy: an **image** is a recipe; a **container** is the dish you cook from it. You can cook many dishes from the same recipe.

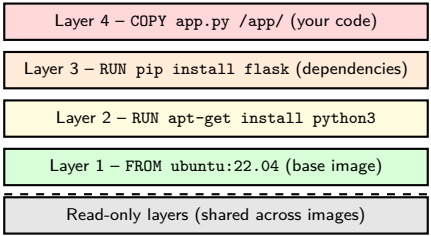
Docker Architecture



The CLI sends commands to the **daemon** via a REST Application Programming Interface (API). The daemon manages images, containers, networks, and volumes.

Docker Images: Layers

- A Docker image is built from **layers**: each instruction in a Dockerfile adds one layer
- Layers are **read-only** and **cached**: only changed layers are rebuilt or re-downloaded



When a container starts, Docker adds a thin **writable layer** on top. All changes (files created, modified) go there. When the container is deleted, that layer is gone.

The Dockerfile

- A **Dockerfile** is a plain text file that describes how to build an image
- Each line is an instruction; Docker executes them top to bottom

```
FROM ubuntu:22.04           # start from base image
RUN apt-get update && \
    apt-get install -y python3 python3-pip   # install deps
WORKDIR /app                # set working directory
COPY app.py .               # copy your code in
RUN pip install flask       # install Python deps
EXPOSE 5000                 # document the port
CMD ["python3", "app.py"]   # default command to run
```

- `FROM`: always the first instruction; defines the base
- `RUN`: executes a shell command during build
- `COPY`: copies files from your machine into the image
- `CMD`: what runs when you do `docker run`

Essential Docker Commands

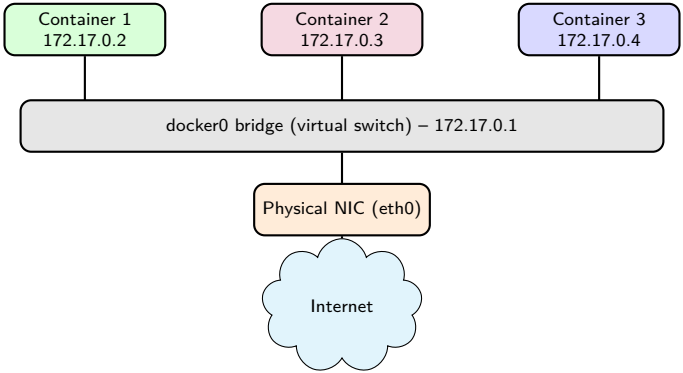
Command	What it does
`docker pull nginx`	Download image from registry
`docker build -t myapp .`	Build image from Dockerfile in current directory
`docker run nginx`	Create and start a container from image
`docker run -d -p 8080:80 nginx`	Run detached, map host port 8080 to container port 80
`docker ps`	List running containers
`docker ps -a`	List all containers (including stopped)
`docker exec -it <id> bash`	Open a shell inside a running container
`docker stop <id>`	Gracefully stop a container
`docker rm <id>`	Delete a stopped container
`docker images`	List locally available images

Outline

upf.

How Containers Connect

- By default, Docker creates a **virtual bridge network** called `docker0`
- Every container gets a **virtual Network Interface Card (vNIC)** connected to that bridge
- The bridge handles Layer 2 forwarding between containers on the same host



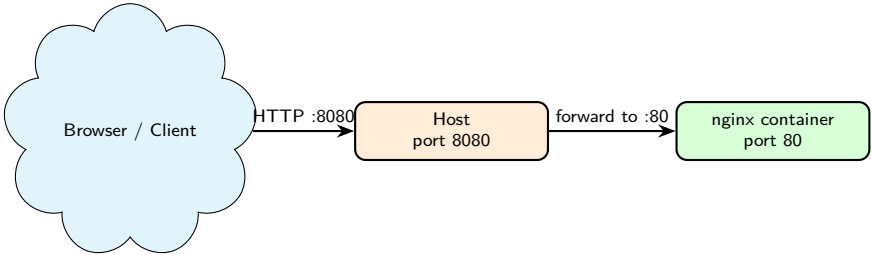
Docker Network Modes

Mode	How it works	Use case
bridge (default)	Container gets private IP on `docker0`; Network Address Translation (NAT) to reach outside	Most single-host scenarios
host	Container shares the host's network stack directly	High-performance / low-latency needs
none	No network interface at all	Isolated batch jobs
custom bridge	User-defined bridge; containers resolve each other by name	Multi-container apps

Port Mapping

- Containers run in an isolated network namespace: **their ports are not visible from outside by default**
- To expose a service, you **map** a host port to a container port:

```
docker run -d -p 8080:80 nginx
```



The Docker daemon intercepts traffic arriving on host port 8080 and forwards it to port 80 inside the container. From the container's perspective, it always listens on port 80.

Outline

upf.

The Problem: Containers Are Ephemeral

- When a container is **deleted**, everything written inside it is lost
- This is intentional: containers are meant to be stateless and replaceable

- **Problem**: some applications need to persist data:
  - Databases (MySQL, PostgreSQL)
  - Log files
  - User-uploaded files

- **Solution**: Docker provides two mechanisms to persist data outside the container:
  - **Volumes**: managed by Docker, stored in `/var/lib/docker/volumes/`
  - **Bind mounts**: map a specific directory from the host into the container

upf.

Volumes vs Bind Mounts

Volume

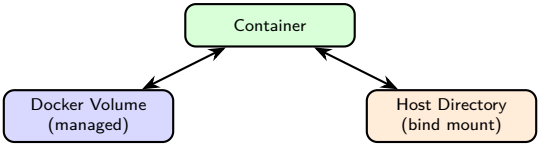
`docker run -v mydata:/data nginx`

- Docker manages the storage location
- Portable across hosts
- Survives container deletion
- Best for **production** use

Bind Mount

`docker run -v /home/user/app:/app nginx`

- You specify the exact host path
- Useful during **development**: edit code on host, see changes live in container
- Less portable (path must exist on host)



Outline

upf.

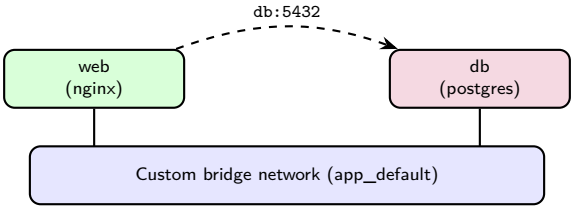
Docker Compose

- Real applications have **multiple services**: web server, database, cache, etc.
- Running them manually with `docker run` is error-prone and hard to reproduce
- **Docker Compose** lets you define all services in a single `docker-compose.yml` file

```yaml
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
```

Docker Compose: How Services Connect

- Compose automatically creates a **custom bridge network** for the application
- Services can reach each other **by name** (DNS resolution built in)



The `web` container can connect to the database simply using `db` as the hostname. No IP addresses needed.

Outline

upf.

Key Takeaways

- **Containers** package an app with its dependencies but share the host kernel: lighter and faster than VMs
- **Docker** is the standard tool to build, ship, and run containers: images, containers, Dockerfile, registry
- Images are built from **layers**: cache makes rebuilds fast
- Docker networking uses a **bridge** by default: containers are isolated but can communicate and expose ports via mapping
- Data written inside a container is **lost on deletion**: use volumes or bind mounts for persistence
- **Docker Compose** manages multi-container applications declaratively

- You are now ready for the lab: `docker pull`, `docker run`, `docker build`, port mapping, and volumes

upf.

References

- Docker Documentation. *Docker Overview*. docs.docker.com
- Docker Documentation. *Networking overview*. docs.docker.com/network
- Docker Documentation. *Use volumes*. docs.docker.com/storage/volumes
- Docker Documentation. *Docker Compose*. docs.docker.com/compose
- Kerrisk, M. *The Linux Programming Interface*. No Starch Press, 2010. (Namespaces and cgroups)